Implémentation des Widgets GTK4
Rapport Technique - Volume 2

Projet GTK Wrapper

March 8, 2026

Contents

1 Introduction **3**

2 Philosophie de conception : Approche Déclarative et C99 **3**

3 Module `window.c` **4**
- 3.1 Fonction `apply_window_background` 4
- 3.2 Fonction `create_window` 5
- 3.3 APIs GTK utilisées 6

4 Module `button.c` **8**
- 4.1 Fonction `create_button` 8
- 4.2 Fonction `create_radio_button` 9
- 4.3 APIs GTK utilisées 10

5 Module `common.c` **11**
- 5.1 Fonction `apply_widget_style` 11
- 5.2 APIs GTK utilisées 12

6 Module `container.c` **14**
- 6.1 Fonction `create_grid` 14
- 6.2 Fonction `create_box` 14
- 6.3 Fonction `create_stack` 15
- 6.4 Helpers complémentaires 16
- 6.5 APIs GTK utilisées 16

7 Module `display.c` **18**
- 7.1 Fonction `create_progress_bar` 18
- 7.2 Fonction `create_spinner` 18
- 7.3 Fonction `create_label` 19
- 7.4 Fonction `create_image` 20
- 7.5 APIs GTK utilisées 20

8 Module `input.c` **22**
8.1 Fonction `create_entry` 22
8.2 Fonction `create_dropdown` 23
8.3 Fonction `create_spin_button` 23
8.4 APIs GTK utilisées 24

9 Module `toggle.c` **26**
9.1 Fonction `create_checkbox` 26
9.2 Fonction `create_switch_row` 26
9.3 APIs GTK utilisées 27

10 Module `date.c` **28**
10.1 Fonction `create_calendar` 28
10.2 APIs GTK utilisées 28

11 Module `menu.c` **30**
11.1 Fonction `register_item_action` 30
11.2 Fonction `append_item_recursive` 30
11.3 Fonctions `create_menubar` et `create_menu_panel` 31
11.4 APIs GTK utilisées 32

12 Module `dialog.c` **33**
12.1 Callback interne `on_alert_response` 33
12.2 Fonction `show_alert_dialog` 33
12.3 APIs GTK utilisées 35

13 Module `separator.c` **36**
13.1 Fonction `create_separator` 36
13.2 APIs GTK utilisées 36

14 Module `demo_app.c` **37**
14.1 Gestion de l'état : `showcase_state` 37
14.2 Callbacks et Logique Métier 37
14.3 Construction de l'Interface 38
14.4 APIs GTK et Logique Appliquée 38

1 Introduction

Ce rapport documente l'implémentation interne du wrapper GTK4. Contrairement au premier volume, centré sur les structures de configuration et l'usage de l'API, ce second document décrit la logique des fonctions, les appels GTK réellement utilisés et les choix techniques effectués dans chaque module.

Les extraits de code présentés ici reprennent l'implémentation réelle du projet. Les commentaires ont été retirés dans les listings afin de conserver une compilation LaTeX simple et robuste tout en gardant un style visuel identique au premier rapport.

2 Philosophie de conception : Approche Déclarative et C99

L'une des innovations majeures de ce wrapper réside dans l'adoption d'un paradigme déclaratif au sein d'un langage procédural comme le C. Au lieu de contraindre le développeur à écrire de multiples appels impératifs (tels que `gtk_window_set_title()`, `gtk_window_set_default_s` etc.), l'API centralise l'état désiré dans des structures de configuration (ex: `window_config`, `button_config`).

Cette approche est rendue particulièrement puissante et expressive grâce à l'utilisation des **initialisateurs désignés (Designated Initializers)** introduits par la norme C99. Cette fonctionnalité permet d'initialiser une structure en nommant explicitement les champs concernés :

```
window_config config = {
    .title = "Mon Application",
    .default_width = 800,
    .resizable = true
};
```

L'utilisation des initialisateurs désignés apporte trois bénéfices fondamentaux à l'architecture du projet :

1. **Lisibilité accrue (Simulation d'arguments nommés) :** Contrairement aux constructeurs C traditionnels nécessitant de passer de nombreux arguments positionnels dont le sens se perd à la lecture, cette syntaxe documente le code de manière inhérente. Le développeur comprend immédiatement quelle propriété est modifiée, s'approchant de l'ergonomie des langages modernes (comme les paramètres nommés en Python ou les objets de configuration en JavaScript).

2. **Sécurité par défaut (Zero-initialization) :** La norme C99 garantit que tout champ non mentionné explicitement dans un bloc d'initialisation désigné est automatiquement initialisé à zéro (les pointeurs deviennent `NULL`, les entiers `0`, et les booléens `false`). Le wrapper exploite cette garantie pour appliquer des valeurs par défaut sûres. Si un développeur omet le champ `.title`, le wrapper détectera un pointeur `NULL` et laissera GTK gérer le comportement par défaut, évitant ainsi les plantages liés aux valeurs résiduelles en mémoire.

3. **Flexibilité et concision :** Les développeurs ne renseignent que les propriétés pertinentes pour leur contexte métier, sans avoir à fournir des valeurs nulles ou factices pour les paramètres dont ils n'ont pas l'utilité.

3 Module `window.c`

Le module `window.c` construit les fenêtres principales de l'application. Il encapsule la création d'un `GtkApplicationWindow`, l'application des propriétés usuelles et la gestion d'un fond personnalisé par CSS.

3.1 Fonction `apply_window_background`

Cette fonction privée applique un style CSS ciblé sur une seule fenêtre. L'idée est de nommer le widget puis de générer dynamiquement une règle CSS qui peut définir soit une couleur de fond seule, soit une couleur combinée à une image.

```c
static void apply_window_background(const window_config *config, GtkWidget *window) {
    if (config == NULL || window == NULL || config->background_color == NULL || config->background_color[0] == '\0') {
        return;
    }

    const char *name = config->widget_name;
    if (name == NULL || name[0] == '\0') {
        name = "wrapper-window";
        gtk_widget_set_name(window, name);
    }

    GtkCssProvider *provider = gtk_css_provider_new();
    char *css;

    if (config->background_image_path != NULL && config->background_image_path[0] != '\0') {
        css = g_strdup_printf(
            "window#%s, window#%s > * {"
            "background-color: %s;"
            "background-image: url('%s');"
            "background-size: cover;"
            "background-position: center;"
            "}",
            name,
            name,
            config->background_color,
            config->background_image_path
        );
    } else {
        css = g_strdup_printf(
            "window#%s, window#%s > * { background-color: %s; }",
            name,
            name,
            config->background_color
        );
    }
```

```

    gtk_css_provider_load_from_string(provider, css);
    gtk_style_context_add_provider_for_display(
        gtk_widget_get_display(window),
        GTK_STYLE_PROVIDER(provider),
        GTK_STYLE_PROVIDER_PRIORITY_APPLICATION
    );

    g_free(css);
    g_object_unref(provider);
}
```

3.2 Fonction `create_window`

La fonction principale du module instancie la fenêtre, applique les propriétés déclarées dans `window_config`, délègue la personnalisation visuelle à `apply_window_background` puis applique le style commun partagé avec les autres widgets.

```
GtkWidget *create_window(GtkApplication *app, const window_config *
   config) {
    GtkWidget *window = gtk_application_window_new(app);
    if (config == NULL) {
        return window;
    }

    if (config->title) {
        gtk_window_set_title(GTK_WINDOW(window), config->title);
    }
    if (config->icon_name) {
        gtk_window_set_icon_name(GTK_WINDOW(window), config->
           icon_name);
    }
    if (config->widget_name) {
        gtk_widget_set_name(window, config->widget_name);
    }

    gtk_window_set_default_size(GTK_WINDOW(window), config->
       default_width, config->default_height);
    gtk_window_set_resizable(GTK_WINDOW(window), config->resizable)
       ;
    gtk_window_set_decorated(GTK_WINDOW(window), config->decorated)
       ;
    gtk_window_set_modal(GTK_WINDOW(window), config->modal);

    if (config->min_width > 0 || config->min_height > 0) {
        gtk_widget_set_size_request(window, config->min_width,
           config->min_height);
    }

    apply_window_background(config, window);
    apply_widget_style(window, &config->style);
```

```

    if (config->maximized) {
        gtk_window_maximize(GTK_WINDOW(window));
    }
    if (config->present_on_create) {
        gtk_window_present(GTK_WINDOW(window));
    }

    return window;
}
```

Macros associées : La macro `CREATE_SIMPLE_WINDOW` illustre parfaitement la synergie entre les macros C et les littéraux composés (compound literals) du C99. En passant directement `&(window_config){.title = t, ...}` en argument, la macro alloue anonymement la structure sur la pile, initialise les champs critiques de manière explicite via les initialisateurs désignés, et transmet son adresse à la fonction constructeur en une seule instruction. Cela évite l'encombrement de la portée locale avec des variables de configuration temporaires, offrant une syntaxe extrêmement compacte et un prototypage accéléré. Les constantes `WIN_SIZE_*` assurent, quant à elles, une cohérence dimensionnelle à travers l'application.

```
#define WIN_SIZE_SMALL  400, 300
#define WIN_SIZE_MEDIUM 800, 600
#define WIN_SIZE_LARGE  1200, 900

#define CREATE_SIMPLE_WINDOW(app, t) create_window(app, &(
    window_config){.title = t, .default_width = 800, .default_height
     = 600, .resizable = true})
```

3.3 APIs GTK utilisées

- `gtk_application_window_new()` : Crée une fenêtre principale intégrée au cycle de vie d'une `GtkApplication`. Le module l'utilise pour bénéficier de la gestion automatique des actions et de la fermeture de l'application.

- `gtk_widget_set_name()` : Assigne un identifiant unique (ID) au widget, utilisable comme sélecteur dans une feuille de style CSS. Le wrapper s'en sert pour cibler spécifiquement la fenêtre lors de l'application dynamique de thèmes personnalisés.

- `gtk_css_provider_new()` : Instancie un objet capable de lire et d'analyser des données CSS. Elle permet au module de transformer les paramètres de couleur et d'image de fond du wrapper en règles de style compréhensibles par le moteur de rendu de GTK.

- `gtk_style_context_add_provider_for_display()` : Attache un fournisseur de style à l'ensemble de l'écran (display). Cette API est cruciale pour que les styles générés dynamiquement par le wrapper soient pris en compte par le moteur CSS de GTK lors du rendu de la fenêtre.

- `gtk_window_set_default_size()` : Détermine les dimensions d'ouverture par défaut de la fenêtre. Elle est utilisée pour initialiser l'espace de travail de l'utilisateur selon les préconisations fournies dans la configuration.
- `gtk_window_present()` : Rend la fenêtre visible et demande au gestionnaire de fenêtres de lui donner le focus. Le wrapper l'appelle systématiquement en fin de création pour garantir que l'utilisateur voit immédiatement l'interface générée.

4 Module button.c

Le module button.c regroupe la création des boutons standards et des boutons radio. Il ajoute au widget GTK de base une logique de thème, d'icône et de raccordement de callbacks.

4.1 Fonction create_button

La fonction create_button traite plusieurs cas d'usage avec une seule interface : bouton texte, bouton avec mnémonique, bouton avec icône, ou combinaison texte plus icône. La variante de thème est appliquée séparément via une fonction interne dédiée.

```c
GtkWidget *create_button(const button_config *config) {
    GtkWidget *button;
    const char *label;

    if (config == NULL) {
        return gtk_button_new();
    }

    label = config->label ? config->label : "";
    button = config->use_underline ? gtk_button_new_with_mnemonic(label) : gtk_button_new_with_label(label);
    gtk_button_set_has_frame(GTK_BUTTON(button), config->has_frame);
    gtk_button_set_can_shrink(GTK_BUTTON(button), config->can_shrink);

    if (config->icon_name && config->icon_name[0] != '\0') {
        GtkWidget *content = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 6);
        GtkWidget *icon = gtk_image_new_from_icon_name(config->icon_name);
        GtkWidget *text = config->use_underline ? gtk_label_new_with_mnemonic(label) : gtk_label_new(label);
        if (config->icon_size > 0) {
            gtk_image_set_pixel_size(GTK_IMAGE(icon), config->icon_size);
        }
        gtk_box_append(GTK_BOX(content), icon);
        gtk_box_append(GTK_BOX(content), text);
        gtk_button_set_child(GTK_BUTTON(button), content);
    }

    apply_theme_variant(button, config->theme_variant);
    apply_widget_style(button, &config->style);

    if (config->on_clicked) {
        g_signal_connect(button, "clicked", G_CALLBACK(config->on_clicked), config->user_data);
```

```
    }

    return button;
}
```

Macros associées : Les macros de variantes de thèmes (`BTN_PRIMARY`, etc.) permettent de s'abstraire des chaînes de caractères brutes lors de la configuration. Bien que simples, elles jouent un rôle crucial pour la fiabilité du code : le compilateur signalera immédiatement une faute de frappe sur le nom de la macro, là où une erreur dans une chaîne littérale comme `"primay"` passerait inaperçue jusqu'à l'exécution, provoquant un rendu silencieusement incorrect de l'interface. Elles garantissent ainsi l'utilisation de noms de classes CSS cohérents avec la logique interne du module.

```
#define BTN_PRIMARY "primary"
#define BTN_SUCCESS "success"
#define BTN_DANGER  "danger"
#define BTN_WARNING "warning"
```

4.2 Fonction `create_radio_button`

Le wrapper radio repose sur `GtkCheckButton`, comme le recommande GTK4. Le groupement exclusif est obtenu en reliant un bouton au précédent du groupe.

```
GtkWidget *create_radio_button(const radio_button_config *config) {
    GtkWidget *button = gtk_check_button_new_with_label(config &&
       config->label ? config->label : "");

    if (config == NULL) {
        return button;
    }

    if (config->group_with != NULL) {
        gtk_check_button_set_group(GTK_CHECK_BUTTON(button),
           GTK_CHECK_BUTTON(config->group_with));
    }
    gtk_check_button_set_active(GTK_CHECK_BUTTON(button), config->
       is_active);
    apply_widget_style(button, &config->style);

    if (config->on_toggled) {
        g_signal_connect(button, "toggled", G_CALLBACK(config->
           on_toggled), config->user_data);
    }

    return button;
}
```

4.3 APIs GTK utilisées

- `gtk_button_new_with_label()` : Crée un nouveau widget bouton contenant un label texte. Dans le module, elle est utilisée pour instancier le bouton de base lorsque l'option `use_underline` est désactivée, offrant une création simplifiée de boutons textuels standards.

- `gtk_button_new_with_mnemonic()` : Crée un bouton dont le label contient un mnémonique (raccourci clavier) indiqué par un caractère souligné. Le wrapper y a recours lorsque `config->use_underline` est vrai, facilitant ainsi l'accessibilité de l'interface au clavier.

- `gtk_button_set_child()` : Définit le widget enfant unique du bouton. Elle permet au wrapper d'injecter une `GtkBox` personnalisée contenant à la fois une icône et un texte, dépassant ainsi la limitation des boutons à simple label.

- `gtk_image_new_from_icon_name()` : Génère un widget d'image à partir du nom d'une icône présente dans le thème système. Elle est utilisée ici pour charger visuellement l'icône demandée dans la configuration avant son insertion dans le bouton.

- `gtk_check_button_set_group()` : Lie un bouton de type case à cocher à un groupe existant pour obtenir un comportement de sélection exclusive. Dans `create_radio_button`, cette API transforme un `GtkCheckButton` standard en un bouton radio fonctionnel.

- `g_signal_connect()` : Connecte une fonction de rappel à un signal spécifique d'un objet. Le module l'utilise pour brancher les signaux `clicked` (pour les boutons) et `toggled` (pour les radios) aux fonctions définies par le développeur applicatif.

5 Module `common.c`

Le module `common.c` fournit la couche de style commune à l'ensemble du wrapper. Il évite la duplication de code en regroupant dans une seule fonction l'application des classes CSS, des marges, de l'alignement, de l'expansion et des métadonnées de confort comme l'infobulle.

5.1 Fonction `apply_widget_style`

La fonction `apply_widget_style` prend un widget GTK déjà créé et lui applique les attributs décrits dans `widget_style_config`. L'implémentation distingue bien les valeurs toujours appliquées, comme les marges, et les valeurs conditionnelles, pilotées par des indicateurs `set_*`.

```c
void apply_widget_style(GtkWidget *widget, const widget_style_config *style) {
    if (widget == NULL || style == NULL) {
        return;
    }

    if (style->css_class && style->css_class[0] != '\0') {
        gtk_widget_add_css_class(widget, style->css_class);
    }
    if (style->css_classes != NULL) {
        for (int i = 0; style->css_classes[i] != NULL; i++) {
            gtk_widget_add_css_class(widget, style->css_classes[i]);
        }
    }

    if (style->width_request != 0 || style->height_request != 0) {
        gtk_widget_set_size_request(widget, style->width_request, style->height_request);
    }

    gtk_widget_set_margin_top(widget, style->margin_top);
    gtk_widget_set_margin_bottom(widget, style->margin_bottom);
    gtk_widget_set_margin_start(widget, style->margin_start);
    gtk_widget_set_margin_end(widget, style->margin_end);

    if (style->set_halign) {
        gtk_widget_set_halign(widget, style->halign);
    }
    if (style->set_valign) {
        gtk_widget_set_valign(widget, style->valign);
    }
    if (style->set_hexpand) {
        gtk_widget_set_hexpand(widget, style->hexpand);
    }
    if (style->set_vexpand) {
        gtk_widget_set_vexpand(widget, style->vexpand);
```

```
    }
    if (style->set_sensitive) {
        gtk_widget_set_sensitive(widget, style->sensitive);
    }
    if (style->set_visible) {
        gtk_widget_set_visible(widget, style->visible);
    }

    if (style->tooltip && style->tooltip[0] != '\0') {
        gtk_widget_set_tooltip_text(widget, style->tooltip);
    }
}
```

Macros associées : L'implémentation de la configuration visuelle est complétée par des macros de substitution. La première série mappe nos propres constantes (ex: `ALIGN_CENTER`) directement sur les types énumérés internes de GTK. Cette approche encapsule le code utilisateur en le découplant légèrement de l'API sous-jacente tout en conservant une résolution à la compilation, assurant une performance maximale sans coût d'exécution. La seconde série (`MARGIN_*`) instaure un système de design (Design System) par paliers fixes. En forçant l'utilisation de ces constantes plutôt que des valeurs magiques éparpillées (magic numbers), le wrapper garantit mécaniquement une harmonie spatiale et une cohérence visuelle dans l'ensemble de l'application.

```
#define ALIGN_START  GTK_ALIGN_START
#define ALIGN_CENTER GTK_ALIGN_CENTER
#define ALIGN_END    GTK_ALIGN_END
#define ALIGN_FILL   GTK_ALIGN_FILL

#define MARGIN_TINY   4
#define MARGIN_SMALL  8
#define MARGIN_MEDIUM 16
#define MARGIN_LARGE  24
```

5.2 APIs GTK utilisées

- `gtk_widget_add_css_class()` : Ajoute une classe de style CSS spécifiée au widget cible. Dans `apply_widget_style`, elle permet d'appliquer de manière dynamique une ou plusieurs classes CSS définies dans la configuration pour modifier l'apparence visuelle du widget.

- `gtk_widget_set_size_request()` : Fixe les dimensions minimales (largeur et hauteur) que le widget doit tenter d'occuper. Le wrapper utilise cette fonction pour forcer une taille spécifique demandée par le développeur dans la structure `widget_style_config`.

- `gtk_widget_set_margin_top()` (et variantes) : Définit les marges extérieures du widget pour chaque direction. Le module les exploite pour assurer un espacement cohérent entre les widgets sans avoir à manipuler directement les conteneurs parents.

- `gtk_widget_set_halign()` et `gtk_widget_set_valign()` : Contrôlent l'alignement du widget respectivement sur les axes horizontal et vertical. Elles permettent au

wrapper de gérer la position du widget (centré, aligné à gauche/droite, ou étendu) dans sa zone d'affichage.

- `gtk_widget_set_hexpand()` et `gtk_widget_set_vexpand()` : Indiquent au gestionnaire de mise en page si le widget doit absorber l'espace supplémentaire disponible. Ces appels sont cruciaux pour définir le comportement de redimensionnement réactif des interfaces construites avec le wrapper.
- `gtk_widget_set_tooltip_text()` : Associe un texte d'aide contextuelle qui apparaît lorsque l'utilisateur survole le widget avec la souris. Elle est systématiquement appelée par le module si une chaîne de caractères `tooltip` est fournie.

6 Module `container.c`

Le module `container.c` rassemble les conteneurs de mise en page du wrapper. Il couvre la grille, la boîte linéaire, la pile de pages et deux helpers complémentaires pour les interfaces multi-vues.

6.1 Fonction `create_grid`

La fonction `create_grid` instancie un `GtkGrid` puis applique l'espacement entre lignes et colonnes ainsi que les options d'homogénéité.

```
GtkWidget *create_grid(const grid_config *config) {
    GtkWidget *grid = gtk_grid_new();
    if (config == NULL) {
        return grid;
    }

    gtk_grid_set_column_spacing(GTK_GRID(grid), config->
       column_spacing);
    gtk_grid_set_row_spacing(GTK_GRID(grid), config->row_spacing);
    gtk_grid_set_column_homogeneous(GTK_GRID(grid), config->
       homogeneous_columns);
    gtk_grid_set_row_homogeneous(GTK_GRID(grid), config->
       homogeneous_rows);
    apply_widget_style(grid, &config->style);
    return grid;
}
```

6.2 Fonction `create_box`

```
GtkWidget *create_box(const box_config *config) {
    GtkWidget *box = gtk_box_new(
        config ? config->orientation : GTK_ORIENTATION_VERTICAL,
        config ? config->spacing : 0
    );
    if (config == NULL) {
        return box;
    }

    gtk_box_set_homogeneous(GTK_BOX(box), config->homogeneous);
    apply_widget_style(box, &config->style);
    return box;
}
```

Macros associées : L'utilisation de macros comme `BOX_H(sp)` et `BOX_V(sp)` démontre la puissance expressive des littéraux composés pour la définition des structures de mise en page. Plutôt que de déclarer une variable nommée de type `box_config` en amont, la macro injecte directement la structure anonyme instanciée au sein même de l'appel de fonction. L'initialisateur désigné garantit que l'orientation est correctement définie sur

le bon axe avec l'espacement requis, tout en laissant implicitement les autres propriétés (comme le style global ou l'homogénéité) à leurs valeurs nulles par défaut (sûres et neutres grâce à la garantie de mise à zéro du C99). Cela réduit drastiquement la verbosité du code de composition d'interface.

```
#define ORIENT_HORIZ GTK_ORIENTATION_HORIZONTAL
#define ORIENT_VERT  GTK_ORIENTATION_VERTICAL

#define BOX_H(sp) (box_config){.orientation = ORIENT_HORIZ, .
    spacing = sp}
#define BOX_V(sp) (box_config){.orientation = ORIENT_VERT, .spacing
     = sp}
```

6.3 Fonction `create_stack`

```
GtkWidget *create_stack(const stack_config *config) {
    GtkWidget *stack = gtk_stack_new();
    if (config == NULL) {
        return stack;
    }

    gtk_stack_set_transition_type(GTK_STACK(stack), config->
        transition);
    gtk_stack_set_transition_duration(GTK_STACK(stack), config->
        duration_ms);
    gtk_stack_set_vhomogeneous(GTK_STACK(stack), config->
        vhomogeneous);
    gtk_stack_set_hhomogeneous(GTK_STACK(stack), config->
        hhomogeneous);
    apply_widget_style(stack, &config->style);
    return stack;
}
```

Macros associées : Les macros de transition et de vitesse permettent de configurer le widget `GtkStack` avec des constantes lisibles et réutilisables.

```
#define TRANSITION_NONE         GTK_STACK_TRANSITION_TYPE_NONE
#define TRANSITION_CROSSFADE    GTK_STACK_TRANSITION_TYPE_CROSSFADE
#define TRANSITION_SLIDE_LEFT
    GTK_STACK_TRANSITION_TYPE_SLIDE_LEFT
#define TRANSITION_SLIDE_RIGHT
    GTK_STACK_TRANSITION_TYPE_SLIDE_RIGHT

#define TRANSITION_FAST   250
#define TRANSITION_NORMAL 400
#define TRANSITION_SLOW   800
```

6.4 Helpers complémentaires

Le module ajoute aussi deux helpers pratiques : `create_stack_sidebar`, qui relie une barre latérale à une pile existante, et `stack_add_page`, qui ajoute une page puis renseigne son titre par propriété GTK.

```
GtkWidget *create_stack_sidebar(GtkWidget *stack, const
    widget_style_config *style) {
    GtkWidget *sidebar = gtk_stack_sidebar_new();
    if (stack != NULL) {
        gtk_stack_sidebar_set_stack(GTK_STACK_SIDEBAR(sidebar),
            GTK_STACK(stack));
    }
    apply_widget_style(sidebar, style);
    return sidebar;
}

void stack_add_page(GtkWidget *stack, GtkWidget *child, const char
    *name, const char *title) {
    if (stack == NULL || child == NULL || name == NULL || title ==
        NULL) {
        return;
    }

    gtk_stack_add_named(GTK_STACK(stack), child, name);
    g_object_set(gtk_stack_get_page(GTK_STACK(stack), child), "
        title", title, NULL);
}
```

6.5 APIs GTK utilisées

- `gtk_grid_new()` : Instancie un conteneur en grille à deux dimensions permettant d'organiser les widgets selon des lignes et des colonnes. Elle est le point de départ de la fonction `create_grid`, qui configure ensuite l'espacement et l'homogénéité.

- `gtk_box_new()` : Crée un conteneur linéaire simple qui dispose ses widgets enfants selon une orientation donnée. Le module l'utilise dans `create_box` pour simplifier la mise en place de flux horizontaux ou verticaux.

- `gtk_stack_new()` : Prépare un conteneur multicouche affichant un seul widget à la fois parmi plusieurs pages. Cette API est au cœur de `create_stack`, facilitant la gestion des transitions et de l'homogénéité des dimensions des pages.

- `gtk_stack_sidebar_new()` : Génère une barre latérale de navigation compatible avec le widget `GtkStack`. Elle est utilisée par le helper correspondant pour fournir un moyen intuitif de changer de page dans une interface à onglets.

- `gtk_stack_add_named()` : Insère un widget dans une pile en lui assignant un nom symbolique unique. Le wrapper l'utilise dans `stack_add_page` pour permettre un référencement aisé de chaque vue de l'application.

- `g_object_set()` : Modifie les propriétés d'un objet via une interface générique à base de chaînes de caractères. Dans le module, elle sert spécifiquement à définir le titre visible d'une page de pile, propriété portée par l'objet `GtkStackPage` interne à GTK.

7 Module `display.c`

Le module `display.c` prend en charge les widgets d'affichage pur : barre de progression, spinner, label et image. Son rôle est d'exposer des constructeurs simples au-dessus des widgets GTK standards.

7.1 Fonction `create_progress_bar`

```c
GtkWidget *create_progress_bar(const progress_bar_config *config) {
    GtkWidget *progress = gtk_progress_bar_new();

    if (config == NULL) {
        return progress;
    }

    gtk_progress_bar_set_fraction(GTK_PROGRESS_BAR(progress),
       config->fraction);
    gtk_progress_bar_set_show_text(GTK_PROGRESS_BAR(progress),
       config->show_text);
    gtk_progress_bar_set_inverted(GTK_PROGRESS_BAR(progress),
       config->inverted);
    gtk_orientable_set_orientation(GTK_ORIENTABLE(progress), config
       ->orientation);
    if (config->text != NULL) {
        gtk_progress_bar_set_text(GTK_PROGRESS_BAR(progress),
           config->text);
    }
    if (config->pulse_once) {
        gtk_progress_bar_pulse(GTK_PROGRESS_BAR(progress));
    }
    apply_widget_style(progress, &config->style);

    return progress;
}
```

Macros associées : Des macros de progression apportent une clarté sémantique sur les valeurs numériques flottantes.

```c
#define PROGRESS_START 0.0
#define PROGRESS_HALF  0.5
#define PROGRESS_FULL  1.0
```

7.2 Fonction `create_spinner`

```c
GtkWidget *create_spinner(const spinner_config *config) {
    GtkWidget *spinner = gtk_spinner_new();

    if (config != NULL) {
```

```
        if (config->size > 0) {
            gtk_widget_set_size_request(spinner, config->size,
               config->size);
        }
        if (config->spinning) {
            gtk_spinner_start(GTK_SPINNER(spinner));
        }
        apply_widget_style(spinner, &config->style);
    }

    return spinner;
}
```

7.3 Fonction `create_label`

```
GtkWidget *create_label(const label_config *config) {
    GtkWidget *label = gtk_label_new(config && config->text ?
       config->text : "");

    if (config == NULL) {
        return label;
    }

    gtk_label_set_selectable(GTK_LABEL(label), config->selectable);
    gtk_label_set_wrap(GTK_LABEL(label), config->wrap);
    gtk_label_set_ellipsize(GTK_LABEL(label), config->ellipsize);
    if (config->max_width_chars > 0) {
        gtk_label_set_max_width_chars(GTK_LABEL(label), config->
           max_width_chars);
    }
    apply_widget_style(label, &config->style);

    return label;
}
```

Macros associées : Ces macros appliquent le concept de l'initialisation désignée et des littéraux composés pour standardiser la typographie. En injectant en ligne une sous-structure imbriquée `.style = {.css_class = "title-1", .margin_bottom = 12}`, la macro configure d'un seul trait le texte et son apparence (hiérarchie CSS et espacement d'aération). Cela évite au programmeur d'éparpiller des constantes de marges ou de noms de classes dans le code de composition de l'application cliente, garantissant ainsi que chaque sous-titre de l'application bénéficiera strictement du même rendu.

```
#define CREATE_TITLE(t)    create_label(&(label_config){.text = t,
   .style = {.css_class = "title-1", .margin_bottom = 12}})
#define CREATE_SUBTITLE(t) create_label(&(label_config){.text = t,
   .style = {.css_class = "title-2", .margin_bottom = 8}})
```

7.4 Fonction `create_image`

```c
GtkWidget *create_image(const image_config *config) {
    GtkWidget *widget;

    if (config == NULL) {
        return gtk_image_new();
    }

    if (config->file_path != NULL && config->file_path[0] != '\0') {
        GtkWidget *pic = gtk_picture_new_for_filename(config->file_path);
        gtk_picture_set_can_shrink(GTK_PICTURE(pic), TRUE);
        gtk_picture_set_content_fit(
            GTK_PICTURE(pic),
            config->keep_aspect ? GTK_CONTENT_FIT_CONTAIN : GTK_CONTENT_FIT_FILL
        );
        widget = pic;
    } else if (config->icon_name != NULL && config->icon_name[0] != '\0') {
        GtkWidget *img = gtk_image_new_from_icon_name(config->icon_name);
        if (config->pixel_size > 0) {
            gtk_image_set_pixel_size(GTK_IMAGE(img), config->pixel_size);
        }
        widget = img;
    } else {
        widget = gtk_image_new();
    }

    apply_widget_style(widget, &config->style);
    return widget;
}
```

7.5 APIs GTK utilisées

- `gtk_progress_bar_set_fraction()` : Définit la fraction de remplissage de la barre de progression, acceptant une valeur flottante entre 0.0 et 1.0. Elle est utilisée dans `create_progress_bar` pour initialiser le niveau d'avancement visuel.
- `gtk_progress_bar_pulse()` : Fait basculer la barre de progression en mode "activité indéterminée", déplaçant un bloc de manière cyclique. Le wrapper l'appelle pour indiquer que le programme est en cours de traitement sans pouvoir estimer le temps restant.
- `gtk_spinner_start()` : Démarre l'animation de rotation du widget spinner. Le module l'utilise pour activer visuellement l'indicateur de chargement dès sa création si la configuration le spécifie.

- `gtk_label_set_ellipsize()` : Détermine le mode de troncature du texte (début, milieu ou fin) par des points de suspension lorsque l'espace est insuffisant. Elle permet au wrapper de garantir une mise en page propre même avec des labels imprévus.
- `gtk_picture_new_for_filename()` : Crée un widget d'image haute performance à partir d'un fichier local. Contrairement à `GtkImage`, cette API utilisée par le module facilite le redimensionnement proportionnel des images chargées.
- `gtk_image_new_from_icon_name()` : Instancie un widget d'image affichant une icône issue du thème de l'utilisateur. Le wrapper l'exploite pour intégrer des symboles graphiques standards dans l'interface de manière simple et légère.

8 Module `input.c`

Le module `input.c` regroupe les composants de saisie. Il encapsule trois widgets principaux : l'entrée de texte, la liste déroulante moderne et le bouton numérique à incréments.

8.1 Fonction `create_entry`

Cette fonction prépare un `GtkEntry` puis applique le placeholder, le texte initial, le type de saisie attendu et les éventuelles contraintes de visibilité ou de longueur.

```c
GtkWidget *create_entry(const entry_config *config) {
    GtkWidget *entry = gtk_entry_new();

    if (config == NULL) {
        return entry;
    }

    if (config->placeholder) {
        gtk_entry_set_placeholder_text(GTK_ENTRY(entry), config->placeholder);
    }
    if (config->default_text) {
        gtk_editable_set_text(GTK_EDITABLE(entry), config->default_text);
    }
    gtk_entry_set_input_purpose(GTK_ENTRY(entry), parse_input_purpose(config->purpose_hint));
    gtk_entry_set_visibility(GTK_ENTRY(entry), !config->password);
    gtk_editable_set_editable(GTK_EDITABLE(entry), config->editable);
    if (config->max_length > 0) {
        gtk_entry_set_max_length(GTK_ENTRY(entry), config->max_length);
    }
    apply_widget_style(entry, &config->style);

    if (config->on_changed) {
        g_signal_connect(entry, "changed", G_CALLBACK(config->on_changed), config->user_data);
    }

    return entry;
}
```

Macros associées : Ces macros de création directe agissent comme un véritable mini-langage dédié (DSL - Domain Specific Language) au-dessus de l'API C du wrapper. Elles exploitent les initialisateurs désignés en ligne pour figer définitivement certains champs sémantiques (comme `.password = true` pour un champ caché ou `.purpose_hint = "email"` pour forcer un clavier spécifique sur mobile) tout en laissant d'autres paramètres (comme le `placeholder`) dynamiques et paramétrables par le développeur. Le résultat

est une déclaration de l'interface utilisateur visuellement très hiérarchisée et presque auto-documentée, rappelant la fluidité des frameworks modernes déclaratifs (tels que SwiftUI, Flutter ou React), où la définition structurelle du widget traduit immédiatement son intention fonctionnelle sans noyer le lecteur dans le paramétrage bas niveau.

```
#define CREATE_PASSWORD_ENTRY(ph) create_entry(&(entry_config){.
    placeholder = ph, .password = true, .purpose_hint = "password"})
#define CREATE_NUMERIC_ENTRY(ph)  create_entry(&(entry_config){.
    placeholder = ph, .purpose_hint = "number"})
#define CREATE_EMAIL_ENTRY(ph)    create_entry(&(entry_config){.
    placeholder = ph, .purpose_hint = "email"})
#define CREATE_URL_ENTRY(ph)      create_entry(&(entry_config){.
    placeholder = ph, .purpose_hint = "url"})
```

8.2 Fonction `create_dropdown`

```
GtkWidget *create_dropdown(const dropdown_config *config) {
    GtkWidget *dropdown;

    if (config == NULL || config->options == NULL) {
        return gtk_drop_down_new(NULL, NULL);
    }

    dropdown = gtk_drop_down_new_from_strings(config->options);
    gtk_drop_down_set_selected(GTK_DROP_DOWN(dropdown), config->
        selected_index);
    gtk_drop_down_set_enable_search(GTK_DROP_DOWN(dropdown), config
        ->enable_search);
    apply_widget_style(dropdown, &config->style);

    if (config->on_selected) {
        g_signal_connect(dropdown, "notify::selected", G_CALLBACK(
            config->on_selected), config->user_data);
    }

    return dropdown;
}
```

8.3 Fonction `create_spin_button`

```
GtkWidget *create_spin_button(const spin_button_config *config) {
    GtkAdjustment *adj;
    GtkWidget *spin;

    if (config == NULL) {
        adj = gtk_adjustment_new(0.0, 0.0, 100.0, 1.0, 5.0, 0.0);
        return gtk_spin_button_new(adj, 1.0, 0);
    }

```

```
    adj = gtk_adjustment_new(config->value, config->min_value,
        config->max_value, config->step, config->page, 0.0);
    spin = gtk_spin_button_new(adj, config->step, config->digits);
    gtk_spin_button_set_numeric(GTK_SPIN_BUTTON(spin), config->
        numeric_only);
    gtk_spin_button_set_wrap(GTK_SPIN_BUTTON(spin), config->wrap);
    apply_widget_style(spin, &config->style);

    if (config->on_value_changed) {
        g_signal_connect(spin, "value-changed", G_CALLBACK(config->
            on_value_changed), config->user_data);
    }

    return spin;
}
```

Macros associées : A l'instar des champs de texte, ces macros pré-configurent le comportement des `SpinButton` pour des cas d'usage typiques (pourcentages, âges, années). Elles masquent la complexité de paramétrage de l'objet `GtkAdjustment` sous-jacent (limites min/max, pas d'incrément, contraintes de décimales) derrière une syntaxe claire et univoque, augmentant considérablement la productivité lors du montage de formulaires de saisie complexes.

```
#define SPIN_PERCENT(v) (spin_button_config){.min_value = 0, .
    max_value = 100, .step = 1, .value = v, .digits = 0}
#define SPIN_AGE(v)     (spin_button_config){.min_value = 0, .
    max_value = 120, .step = 1, .value = v, .digits = 0}
#define SPIN_YEAR(v)    (spin_button_config){.min_value = 1900, .
    max_value = 2100, .step = 1, .value = v, .digits = 0}
```

8.4 APIs GTK utilisées

- `gtk_entry_set_placeholder_text()` : Affiche un message d'incitation à l'intérieur du champ de saisie tant que celui-ci est vide. Le module l'utilise pour fournir des indications contextuelles à l'utilisateur sans encombrer l'interface avec des labels externes.

- `gtk_entry_set_input_purpose()` : Indique la nature des données attendues (URL, email, chiffres, etc.) pour optimiser le comportement des méthodes de saisie, comme le clavier virtuel sur mobile. Le wrapper l'utilise pour améliorer l'ergonomie des formulaires.

- `gtk_drop_down_new_from_strings()` : Construit une liste déroulante moderne en utilisant un tableau de chaînes de caractères pour les options. Cette API simplifie grandement la création de menus de sélection dans le module par rapport aux anciens widgets GTK.

- `gtk_drop_down_set_enable_search()` : Permet d'intégrer une barre de filtrage rapide à l'intérieur du menu déroulant. Le wrapper active cette fonctionnalité pour faciliter la sélection lorsque le nombre d'options devient important.

- `gtk_adjustment_new()` : Définit un modèle de données numérique spécifiant une valeur courante, des bornes minimale et maximale, ainsi que des pas de progression. Elle constitue la fondation logique du widget `GtkSpinButton` créé par le module.
- `gtk_spin_button_new()` : Crée le widget de contrôle numérique lié à un ajustement. Elle permet à l'utilisateur de saisir ou d'incrémenter précisément une valeur numérique dans les limites définies par le wrapper.

9 Module `toggle.c`

Le module `toggle.c` couvre les composants binaires du wrapper : case à cocher et ligne composée avec interrupteur. Ces deux widgets reposent sur des contrôles GTK simples, mais le wrapper leur ajoute une configuration déclarative homogène.

9.1 Fonction `create_checkbox`

```c
GtkWidget *create_checkbox(const checkbox_config *config) {
    GtkWidget *checkbox = gtk_check_button_new_with_label(config &&
        config->label ? config->label : "");

    if (config == NULL) {
        return checkbox;
    }

    gtk_check_button_set_active(GTK_CHECK_BUTTON(checkbox), config
       ->active);
    gtk_check_button_set_inconsistent(GTK_CHECK_BUTTON(checkbox),
       config->inconsistent);
    apply_widget_style(checkbox, &config->style);

    if (config->on_toggled != NULL) {
        g_signal_connect(checkbox, "toggled", G_CALLBACK(config->
           on_toggled), config->user_data);
    }

    return checkbox;
}
```

9.2 Fonction `create_switch_row`

Cette seconde fonction construit un composant composite : un label à gauche, un `GtkSwitch` à droite, puis un éventuel callback sur la propriété `active`.

```c
GtkWidget *create_switch_row(const switch_config *config, GtkWidget
    **out_switch) {
    GtkWidget *row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    GtkWidget *label = gtk_label_new(config && config->label ?
       config->label : "");
    GtkWidget *sw = gtk_switch_new();

    gtk_widget_set_hexpand(label, TRUE);
    gtk_widget_set_halign(label, GTK_ALIGN_START);
    gtk_widget_set_halign(sw, GTK_ALIGN_END);
    gtk_box_append(GTK_BOX(row), label);
    gtk_box_append(GTK_BOX(row), sw);

    if (config != NULL) {
        gtk_switch_set_active(GTK_SWITCH(sw), config->active);
```

```
        gtk_switch_set_state(GTK_SWITCH(sw), config->state);
        apply_widget_style(row, &config->style);

        if (config->on_active_changed != NULL) {
            g_signal_connect(sw, "notify::active", G_CALLBACK(
               config->on_active_changed), config->user_data);
        }
    }

    if (out_switch != NULL) {
        *out_switch = sw;
    }

    return row;
}
```

9.3 APIs GTK utilisées

- `gtk_check_button_new_with_label()` : Initialise une case à cocher associée à un texte descriptif. Le module l'utilise pour créer des contrôles de sélection binaire simples et lisibles.

- `gtk_check_button_set_inconsistent()` : Permet de placer visuellement la case à cocher dans un état tiers, indiquant une sélection partielle ou indéterminée. Le wrapper expose cette fonctionnalité pour gérer des logiques de sélection complexes.

- `gtk_switch_new()` : Instancie un widget d'interrupteur coulissant, typique des interfaces modernes. Elle est au cœur de la fonction `create_switch_row` pour proposer une interaction intuitive sur les écrans tactiles ou avec la souris.

- `gtk_switch_set_active()` : Modifie l'état booléen du switch sans déclencher de signaux de changement d'état. Le wrapper l'utilise pour appliquer l'état initial défini par le développeur lors de la création du widget.

- `gtk_box_append()` : Insère un widget à la suite des autres dans un conteneur `GtkBox`. Dans le module, elle sert à construire dynamiquement la ligne composite regroupant le libellé et l'interrupteur.

- `g_signal_connect()` : Raccorde les signaux de changement d'état (comme `notify::active` pour le switch) aux callbacks applicatifs. Elle assure la réactivité de l'application aux changements d'état opérés par l'utilisateur sur les contrôles de bascule.

10 Module `date.c`

Le module `date.c` encapsule le widget calendrier de GTK4. Il prend en charge la date initiale, l'affichage des noms de jours, les numéros de semaine et la remontée du signal de sélection.

10.1 Fonction `create_calendar`

```c
GtkWidget *create_calendar(const calendar_config *config) {
    GtkWidget *calendar = gtk_calendar_new();

    if (config == NULL) {
        return calendar;
    }

    if (config->selected_date != NULL) {
#if GTK_CHECK_VERSION(4, 20, 0)
        gtk_calendar_set_date(GTK_CALENDAR(calendar), config->selected_date);
#else
        gtk_calendar_select_day(GTK_CALENDAR(calendar), config->selected_date);
#endif
    }
    gtk_calendar_set_show_day_names(GTK_CALENDAR(calendar), config->show_day_names);
    gtk_calendar_set_show_week_numbers(GTK_CALENDAR(calendar), config->show_week_numbers);
    gtk_calendar_set_show_heading(GTK_CALENDAR(calendar), !config->no_month_change);
    apply_widget_style(calendar, &config->style);

    if (config->on_day_selected != NULL) {
        g_signal_connect(calendar, "day-selected", G_CALLBACK(config->on_day_selected), config->user_data);
    }

    return calendar;
}
```

10.2 APIs GTK utilisées

- `gtk_calendar_new()` : Instancie le composant de base du calendrier GTK4. Cette API initialise le widget permettant l'affichage et la sélection interactive de dates dans le module.

- `gtk_calendar_set_date()` : Positionne le calendrier sur une date précise (jour, mois, année) à l'aide d'un objet `GDateTime`. Disponible depuis GTK 4.20, elle est privilégiée pour définir l'état initial du widget selon la configuration.

- `gtk_calendar_select_day()` : Remplit le même rôle que l'API précédente mais pour les versions de GTK4 plus anciennes. Son utilisation dans le module garantit la portabilité du code sur différents environnements système.
- `gtk_calendar_set_show_week_numbers()` : Bascule l'affichage des colonnes de numérotation des semaines sur le côté gauche du widget. Elle est directement pilotée par le booléen `show_week_numbers` de la structure de configuration.
- `gtk_calendar_set_show_heading()` : Contrôle la visibilité de la barre supérieure affichant le mois et l'année. Le wrapper s'en sert pour activer ou désactiver la navigation mensuelle directe par l'utilisateur.
- `g_signal_connect()` : Attache un callback au signal `day-selected`. Cette connexion permet au wrapper de notifier le reste de l'application dès qu'une nouvelle date est cliquée sur le calendrier.

11 Module menu.c

Le module menu.c implémente l'architecture moderne des menus GTK4. Les éléments visibles sont décrits via GMenuModel, tandis que la logique applicative est fournie sous forme d'actions enregistrées dans un GActionMap.

11.1 Fonction register_item_action

Cette fonction privée crée une action GTK pour une entrée de menu active, connecte le callback utilisateur puis enregistre l'action dans la map de l'application.

```
static void register_item_action(const menubar_config *config,
                                 const menu_item_config *item,
                                 const char *action_name) {
    if (config == NULL || config->action_map == NULL || item ==
       NULL || item->on_activate == NULL || action_name == NULL) {
        return;
    }

    GSimpleAction *action = g_simple_action_new(action_name, NULL);
    g_signal_connect(action, "activate", G_CALLBACK(item->
       on_activate), item->user_data);
    g_action_map_add_action(config->action_map, G_ACTION(action));
    g_object_unref(action);
}
```

11.2 Fonction append_item_recursive

La construction du menu est récursive. Le code traite d'abord les sous-menus et les sections, puis les actions simples en leur attribuant un nom unique.

```
static void append_item_recursive(GMenu *menu,
                                  const menu_item_config *item,
                                  const menubar_config *config) {
    if (item == NULL || item->enabled == false) {
        return;
    }

    if (item->kind == MENU_ITEM_SUBMENU && item->children != NULL
       && item->child_count > 0) {
        GMenu *submenu = g_menu_new();
        for (size_t i = 0; i < item->child_count; i++) {
            append_item_recursive(submenu, &item->children[i],
               config);
        }
        g_menu_append_submenu(menu, item->label ? item->label : "",
            G_MENU_MODEL(submenu));
        g_object_unref(submenu);
        return;
    }

```

```
    if (item->kind == MENU_ITEM_SECTION && item->children != NULL
       && item->child_count > 0) {
        GMenu *section = g_menu_new();
        for (size_t i = 0; i < item->child_count; i++) {
            append_item_recursive(section, &item->children[i],
               config);
        }
        g_menu_append_section(menu, item->label, G_MENU_MODEL(
           section));
        g_object_unref(section);
        return;
    }

    char *action_name = create_action_name();
    register_item_action(config, item, action_name);

    char *detailed_action = g_strdup_printf("app.%s", action_name);
    g_menu_append(menu, item->label ? item->label : "",
       detailed_action);

    g_free(detailed_action);
    g_free(action_name);
}
```

11.3 Fonctions `create_menubar` et `create_menu_panel`

Les fonctions publiques consomment un modèle construit par `build_menu_model`. La première fabrique une barre de menus, la seconde un panneau de menu réutilisable dans un popover.

```
GtkWidget *create_menubar(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *bar = gtk_popover_menu_bar_new_from_model(model);
    if (config != NULL) {
        apply_widget_style(bar, &config->style);
    }
    g_object_unref(model);
    return bar;
}

GtkWidget *create_menu_panel(const menubar_config *config) {
    GMenuModel *model = build_menu_model(config);
    GtkWidget *panel = gtk_popover_menu_new_from_model(model);

    if (config != NULL) {
        gtk_orientable_set_orientation(
            GTK_ORIENTABLE(panel),
            config->layout == MENU_LAYOUT_VERTICAL ?
               GTK_ORIENTATION_VERTICAL :
               GTK_ORIENTATION_HORIZONTAL
        );
```

```
        gtk_popover_set_has_arrow(GTK_POPOVER(panel), config->
            show_arrow_indicators);
        apply_widget_style(panel, &config->style);
    }

    g_object_unref(model);
    return panel;
}
```

11.4 APIs GTK utilisées

- `g_simple_action_new()` : Crée un objet d'action élémentaire identifié par un nom unique. Le module l'utilise pour définir l'entité logique qui sera déclenchée lors de la sélection d'un item, assurant une séparation propre entre l'interface et le code métier.

- `g_action_map_add_action()` : Enregistre une action dans la carte des actions de l'application ou de la fenêtre. Cette API permet au wrapper de rendre les items de menu fonctionnels en les rendant accessibles au système d'actions global de GTK.

- `g_menu_append_submenu()` : Ajoute une entrée de type sous-menu pointant vers un autre modèle de menu. Elle est la clé de la construction récursive utilisée dans le module pour gérer des structures de menus imbriquées de n'importe quelle profondeur.

- `g_menu_append_section()` : Permet de regrouper des items de menu dans une zone visuellement délimitée (souvent avec un titre). Le wrapper l'exploite pour organiser l'interface de manière hiérarchique et logique.

- `gtk_popover_menu_bar_new_from_model()` : Génère une barre de menu complète en se basant sur la structure décrite dans un `GMenuModel`. C'est l'API principale pour transformer la configuration déclarative du wrapper en un widget de barre de menu GTK4.

- `gtk_popover_menu_new_from_model()` : Produit un widget de menu adapté à un affichage dans une fenêtre flottante (popover). Le module l'utilise pour offrir des menus contextuels ou des panneaux de contrôle compacts basés sur le même modèle que la barre de menu.

12 Module `dialog.c`

Le module `dialog.c` implémente les boîtes d'alerte asynchrones. Il s'appuie sur `GtkAlertDialog`, qui ne bloque pas l'exécution du programme et impose donc un traitement différé de la réponse utilisateur.

12.1 Callback interne `on_alert_response`

Cette fonction récupère le résultat asynchrone, transforme une éventuelle erreur en indice `-1`, puis transmet la réponse au callback utilisateur avant de libérer les ressources temporaires.

```c
static void on_alert_response(GObject *source_object, GAsyncResult
    *result, gpointer user_data) {
    (void)source_object;
    alert_async_context *ctx = user_data;
    GError *error = NULL;
    int index = gtk_alert_dialog_choose_finish(ctx->dialog, result,
        &error);
    if (error != NULL) {
        g_error_free(error);
        index = -1;
    }
    if (ctx->on_response != NULL) {
        ctx->on_response(index, ctx->user_data);
    }
    g_object_unref(ctx->dialog);
    g_free(ctx);
}
```

12.2 Fonction `show_alert_dialog`

La fonction publique configure le message, les détails, la modalité et la liste des boutons. Elle déduit aussi le bouton par défaut et le bouton d'annulation à partir de la configuration avant d'afficher le dialogue en mode synchrone visuel mais asynchrone du point de vue du code.

```c
void show_alert_dialog(const alert_dialog_config *config) {
    GtkAlertDialog *dialog;
    const char **buttons;

    if (config == NULL || config->message == NULL) {
        return;
    }

    dialog = gtk_alert_dialog_new("%s", config->message);
    if (config->detail != NULL) {
        gtk_alert_dialog_set_detail(dialog, config->detail);
    }
    gtk_alert_dialog_set_modal(dialog, config->modal);

    if (config->button_count > 0 && config->buttons != NULL) {
```

```c
        buttons = g_new0(const char *, config->button_count + 1);
        for (size_t i = 0; i < config->button_count; i++) {
            buttons[i] = config->buttons[i].label;
        }
        gtk_alert_dialog_set_buttons(dialog, buttons);
        g_free(buttons);
    }

    int default_button = config->default_button;
    int cancel_button = config->cancel_button;

    if (config->buttons != NULL && config->button_count > 0) {
        for (size_t i = 0; i < config->button_count; i++) {
            const char *appearance = config->buttons[i].appearance;
            if (appearance == NULL) {
                continue;
            }
            if (default_button < 0 && strcmp(appearance, "suggested") == 0) {
                default_button = (int)i;
            }
            if (cancel_button < 0 && strcmp(appearance, "cancel") == 0) {
                cancel_button = (int)i;
            }
        }
    }

    if (default_button >= 0) {
        gtk_alert_dialog_set_default_button(dialog, default_button);
    }
    if (cancel_button >= 0) {
        gtk_alert_dialog_set_cancel_button(dialog, cancel_button);
    }

    if (config->on_response != NULL) {
        alert_async_context *ctx = g_new0(alert_async_context, 1);
        ctx->dialog = g_object_ref(dialog);
        ctx->on_response = config->on_response;
        ctx->user_data = config->user_data;
        gtk_alert_dialog_choose(dialog, config->parent, NULL, on_alert_response, ctx);
    } else {
        gtk_alert_dialog_show(dialog, config->parent);
    }

    g_object_unref(dialog);
}
```

12.3 APIs GTK utilisées

- `gtk_alert_dialog_new()` : Instancie un nouvel objet de dialogue d'alerte en acceptant une chaîne de format pour le message principal. Elle est systématiquement appelée au début de `show_alert_dialog` pour définir le cœur de l'information à transmettre.
- `gtk_alert_dialog_set_buttons()` : Assigne un tableau de libellés aux boutons qui seront présentés à l'utilisateur. Le wrapper convertit sa liste interne de structures de boutons vers ce format simple attendu par GTK4.
- `gtk_alert_dialog_set_default_button()` : Désigne l'indice du bouton qui doit répondre à l'activation par défaut (touche Entrée). Cela permet d'améliorer l'ergonomie en mettant en avant l'action la plus probable ou la plus sûre.
- `gtk_alert_dialog_choose()` : Lance l'affichage du dialogue de manière non bloquante et enregistre un callback pour le futur. Elle est essentielle pour le fonctionnement asynchrone du wrapper, permettant à l'application de continuer à tourner pendant que le dialogue est ouvert.
- `gtk_alert_dialog_choose_finish()` : Extrait le résultat de l'interaction (l'indice du bouton cliqué) à partir de l'objet de résultat asynchrone. Elle est invoquée dans `on_alert_response` pour clore proprement le cycle de vie du dialogue.
- `gtk_alert_dialog_show()` : Affiche le dialogue à l'écran sans mettre en place de mécanisme de capture de la réponse. Le module y a recours pour les simples alertes informatives où seule la fermeture manuelle par l'utilisateur compte.

13 Module separator.c

Le module separator.c est volontairement minimal. Il fournit un simple constructeur pour les séparateurs visuels, tout en restant cohérent avec le reste du wrapper grâce à widget_style_config.

13.1 Fonction create_separator

```
GtkWidget *create_separator(const separator_config *config) {
    GtkWidget *separator = gtk_separator_new(config ? config->
       orientation : GTK_ORIENTATION_HORIZONTAL);
    if (config != NULL) {
        apply_widget_style(separator, &config->style);
    }
    return separator;
}
```

13.2 APIs GTK utilisées

- gtk_separator_new() : Instancie une ligne de séparation visuelle dont l'orientation (horizontale ou verticale) est définie à la création. Le module l'utilise pour fournir un moyen simple de segmenter l'interface graphique tout en appliquant le style commun du wrapper.

14 Module demo_app.c

Le module demo_app.c constitue le cœur de l'application de démonstration. Il orchestre l'assemblage des widgets via les structures de configuration du wrapper, gère l'état de l'interface et définit la logique de réaction aux événements utilisateur.

14.1 Gestion de l'état : showcase_state

Pour maintenir une cohérence entre les différents composants (par exemple, mettre à jour un label de statut depuis un changement dans une entrée de texte), l'application utilise une structure d'état centralisée.

```c
typedef struct {
  GtkWidget *status_label;
  GtkWidget *progress;
  GtkWidget *spinner;
  GtkWidget *switch_widget;
  bool spinner_running;
} showcase_state;
```

Cette structure est allouée dynamiquement lors du démarrage de l'application et passée en tant que user_data à tous les callbacks.

14.2 Callbacks et Logique Métier

L'application implémente de nombreux callbacks pour démontrer l'interactivité. Par exemple, la mise à jour du statut lors de la saisie :

```c
static void on_entry_changed(GtkEditable *editable, gpointer
    user_data) {
  showcase_state *state = user_data;
  char msg[96];
  g_snprintf(msg, sizeof(msg), "Input length: %zu",
             strlen(gtk_editable_get_text(editable)));
  set_status(state, msg);
}
```

Ou encore le pilotage asynchrone des boîtes de dialogue :

```c
static void on_show_dialog(GtkButton *button, gpointer user_data) {
  showcase_state *state = user_data;
  GtkRoot *root = gtk_widget_get_root(state->status_label);
  alert_button_config buttons[] = {
      {.label = "Cancel", .appearance = "flat"},
      {.label = "Confirm", .appearance = "suggested"},
  };

  show_alert_dialog(&(alert_dialog_config){
      .parent = (root && GTK_IS_WINDOW(root)) ? GTK_WINDOW(root) :
         NULL,
      .message = "Proceed with wrapper-driven action?",
      .buttons = buttons,
```

```
        .button_count = G_N_ELEMENTS(buttons),
        .on_response = on_alert_result,
        .user_data = state,
    });
}
```

14.3 Construction de l'Interface

L'interface est découpée en "pages" intégrées dans un `GtkStack`. Chaque page est construite par une fonction dédiée (`make_inputs_page`, `make_buttons_page`, etc.) qui utilise les fonctions `create_*` du wrapper.

La fonction `build_showcase` assemble ces pages, ajoute une barre latérale de navigation (`GtkStackSidebar`), un menu principal (`GtkPopoverMenuBar`) et une barre de statut.

14.4 APIs GTK et Logique Appliquée

- `gtk_application_get_active_window()` : Permet de récupérer la fenêtre actuellement focalisée pour servir de parent aux boîtes de dialogue surgissantes.
- `gtk_widget_get_root()` : Utilisé pour remonter la hiérarchie des widgets à partir du label de statut afin de trouver la fenêtre racine.
- `g_snprintf()` : Employé systématiquement pour formater les chaînes de caractères de statut de manière sécurisée (prévention des dépassements de tampon).
- `g_new0()` : Alloue la structure `showcase_state` en initialisant tous ses champs à zéro, garantissant qu'aucun pointeur n'est indéfini au démarrage.
- `gtk_window_set_child()` : Définit le conteneur principal (`root`) comme contenu unique de la fenêtre d'application.
- `g_signal_connect()` : Point central de l'interactivité, reliant les signaux de bas niveau (clic, modification) aux fonctions de haut niveau de la démonstration.